

Lekcja

Temat: Ćwiczenia praktyczne tworzenia bazy danych, tabel oraz wypełniania danymi

Zadanie 1 – Biblioteka

Utwórz bazę danych **biblioteka**.

Następnie utwórz tabelę **ksiazki** zawierającą następujące kolumny:

Kolumna	Typ danych
id	INT
tytul	VARCHAR(100)
autor	VARCHAR(100)
rok_wydania	YEAR
liczba_stron	INT
cena	DECIMAL(8,2)
dostepna	BOOLEAN

Polecenia

1. Utwórz bazę danych.
2. Utwórz tabelę ksiazki.
3. Ustaw kolumnę id jako klucz główny z automatycznym numerowaniem.
4. Dodaj samodzielnie minimum 10 książek.
5. Wyświetl wszystkie rekordy.
6. Wyświetl tylko książki droższe niż 50 zł.
7. Posortuj książki według roku wydania malejąco.

Zadanie 2 – Sklep internetowy

Utwórz bazę danych sklep.

Następnie utwórz tabelę **produkty** zawierającą następujące kolumny:

Kolumna	Typ danych
id	INT
nazwa	VARCHAR(100)
kategoria	VARCHAR(100)
cena	DECIMAL(10,2)
stan_magazynowy	INT
producent	VARCHAR(100)
data_dodania	DATE

Polecenia

1. Utwórz tabelę.
2. Wprowadź samodzielnie co najmniej 15 produktów.

3. Wyświetl produkty z kategorii „Elektronika”.
4. Wyświetl produkty z ceną pomiędzy 100 a 500.
5. Posortuj produkty według ceny rosnąco.
6. Policz liczbę produktów każdego producenta.

Lekcja

Temat: Ćwiczenia praktyczne z metodami i funkcjami tekstowymi w MySQL

```
CREATE TABLE users (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    first_name VARCHAR(20),  
    last_name VARCHAR(20),  
    email VARCHAR(50),  
    city VARCHAR(20),  
    description VARCHAR(10)  
);  
  
INSERT INTO users(first_name, last_name, email, city, description) VALUES  
(  
    'Jan', 'Kowalski', 'jan@gmail.com', 'Warszawa', 'Programista'),  
(  
    'Anna', 'Nowak', 'anna@gmail.com', 'Kraków', 'Tester'),  
(  
    'Piotr', 'Wiśniewski', NULL, 'Gdańsk', NULL),  
(  
    'Maria', NULL, 'maria@gmail.com', NULL, 'DevOps');
```

1. CONCAT()

Łączy kilka tekstów w jeden ciąg znaków.

```
SELECT CONCAT(first_name, ' ', last_name) AS full_name FROM users;
```

Zachowanie dla NULL

Jeżeli choć jeden argument jest NULL, cały wynik jest NULL.

```
SELECT CONCAT('Jan', NULL);
```

Wskazówka: obejście

```
SELECT CONCAT(first_name, ' ', IFNULL(last_name, 'BRAK')) FROM users;
```

2. CONCAT_WS()

WS = With Separator.

Łączy teksty używając wskazanego separatora i pomija wartości NULL.

```
SELECT CONCAT_WS(' - ', first_name, last_name) FROM users;
```

Ciekawostka

NULL jest pomijany.

```
SELECT CONCAT_WS('-', 'A', NULL, 'B');
```

3. LENGTH()

Zwraca liczbę bajtów zajmowanych przez tekst.

```
SELECT LENGTH('ABC');
```

Polskie znaki:

```
SELECT LENGTH('ą');
```

W UTF-8: 2 lub nawet 4 bajty zależnie od kodowania.

4. CHAR_LENGTH()

Zwraca liczbę znaków w tekście niezależnie od kodowania.

```
SELECT CHAR_LENGTH('ą');
```

5. UPPER() / LOWER()

Zamienia wszystkie litery na wielkie/ Zamienia wszystkie litery na małe.

```
SELECT UPPER(first_name) FROM users;
```

6. SUBSTRING()

Wycina fragment tekstu od wskazanej pozycji.

SUBSTRING(tekst, start, długość)

- start > 0 → liczenie od początku
- start < 0 → liczenie od końca
- długość opcjonalna

```
SELECT SUBSTRING('1234567890abcdefghij', 1, 3);
```

```
SELECT SUBSTRING('1234567890abcdefghij', 4);
```

```
SELECT SUBSTRING('1234567890abcdefghij', -7, 4);
```

```
SELECT SUBSTRING('1234567890abcdefghij', 4, -2);
```

```
SELECT SUBSTRING('1234567890abcdefghij', NULL, 3)
```

```
SELECT SUBSTRING(NULL, 1, 2);
```

7. LEFT() i RIGHT()

Pobiera określoną liczbę znaków od początku tekstu. / Pobiera określoną liczbę znaków od końca tekstu.

```
SELECT LEFT('1234567890', 2);
```

```
SELECT LEFT('1234567890', -2);
```

```
SELECT RIGHT('1234567890', 2);
```

```
SELECT RIGHT('1234567890', -2);
```

8. TRIM()

Usuwa spacje z początku i końca tekstu.

```
SELECT TRIM(' Jan ');
```

LTRIM()

Usuwa spacje tylko z początku tekstu.

```
SELECT LTRIM(' Jan');
```

RTRIM()

Usuwa spacje tylko z końca tekstu.

```
SELECT RTRIM('Jan ');
```

9. REPLACE()

Zamienia wskazany fragment tekstu na inny.

```
SELECT REPLACE(email, 'gmail', 'wp') FROM users;
```

10. REVERSE()

Odwraca kolejność znaków w tekście.

```
SELECT REVERSE('MYSQL');
```

11. LOCATE()

Zwraca pozycję pierwszego wystąpienia szukanego ciągu.

```
SELECT LOCATE('@', email) FROM users;
```

12. INSTR()

Wyszukuje ciąg znaków i zwraca jego pozycję w tekście.

```
SELECT INSTR(email, '@') FROM users;
```

13. POSITION()

Standardowa SQL-owa wersja funkcji wyszukiującej pozycję tekstu.

```
SELECT POSITION('@' IN email) FROM users;
```

14. LPAD()

Dopełnia tekst z lewej strony wskazanymi znakami do określonej długości.

```
SELECT LPAD('123', 5, '0');
```

15. RPAD()

Dopełnia tekst z prawej strony wskazanymi znakami do określonej długości.

```
SELECT RPAD('ABC', 6, '*');
```

16. REPEAT()

Powtarza tekst określoną liczbę razy.

```
SELECT REPEAT('*', 5);
```

17. SPACE()

Tworzy ciąg składający się z podanej liczby spacji.

```
SELECT CONCAT('A',SPACE(5),'B');
```

18. ASCII()

Zwraca kod ASCII pierwszego znaku tekstu.

```
SELECT ASCII('A');
```

19. CHAR()

Zamienia kod ASCII na odpowiadający mu znak.

```
SELECT CHAR(65);
```

20. FORMAT()

Formatuje liczbę z określoną liczbą miejsc po przecinku oraz separatorami tysięcy.

```
SELECT FORMAT(1234567.89,2);
```

```
SELECT FORMAT(1234567.89754,2);
```

21. ELT()

Zwraca numer pozycji szukanej wartości na liście argumentów.

Pobiera element z listy.

```
SELECT ELT(2,'Java','SQL','Angular');
```

```
SELECT ELT(3,'Java','SQL','Angular');
```

```
SELECT ELT(63,'Java','SQL','Angular');
```

Lekcja

Temat: ALERT w MySQL

Polecenie ALTER TABLE służy do **modyfikowania istniejącej tabeli** bez jej usuwania. Możesz zmieniać strukturę tabeli już po jej utworzeniu.

Przykładowa tabela:

```
CREATE TABLE Pracownik (  
    id INT PRIMARY KEY,  
    imie VARCHAR(50)  
);
```

Po utworzeniu tabeli możesz ją modyfikować za pomocą ALTER TABLE.

Najczęstsze zastosowania

1. Dodawanie kolumn

```
ALTER TABLE Pracownik  
ADD nazwisko VARCHAR(50);
```

Przed:

id	imie
1	Jan

Po:

id	imie	nazwisko
1	Jan	NULL

2. Usuwanie kolumn

ALTER TABLE Pracownik

DROP COLUMN nazwisko;

Usuwa kolumnę wraz z danymi.

3. Zmiana typu danych kolumny

ALTER TABLE pracownik

MODIFY COLUMN imie VARCHAR(100);

SQL Server

ALTER TABLE pracownik

ALTER COLUMN imie VARCHAR(100);

Zmiana z 50 na 100 znaków.

4. Usuwanie i dodawanie klucza głównego (PRIMARY KEY)

ALTER TABLE pracownik

DROP PRIMARY KEY;

ALTER TABLE Pracownik

ADD CONSTRAINT PK_Pracownik PRIMARY KEY (id);

5. Dodawanie klucza obcego (FOREIGN KEY)

Tabele:

CREATE TABLE Dzial (

id INT PRIMARY KEY,

nazwa VARCHAR(50)

);

CREATE TABLE Osoba (

id INT PRIMARY KEY,

imie VARCHAR(50), dzial_id INT

);

Dodanie relacji:

```
ALTER TABLE Osoba  
ADD CONSTRAINT FK_Pracownik_Dzial  
FOREIGN KEY (dzial_id)  
REFERENCES Dzial(id);
```

Od tego momentu osoba może wskazywać tylko istniejący dział.

6. Dodawanie ograniczenia NOT NULL

Jeśli kolumna istnieje:

```
ALTER TABLE pracownik  
MODIFY COLUMN imie VARCHAR(50) NOT NULL;  
Od tej chwili pole musi zawierać wartość.
```

7. Dodawanie wartości domyślnej

```
ALTER TABLE Pracownik  
MODIFY COLUMN imie VARCHAR(50) DEFAULT 'Nieznane';
```

Przy insercie:

```
INSERT INTO Pracownik(id) VALUES (1);
```

zostanie zapisane:

```
Select * from Pracownik;
```

id = 1

imie = 'Nieznane'

8. Dodawanie ograniczenia CHECK

```
ALTER TABLE Pracownik  
ADD COLUMN wiek INT;  
ALTER TABLE Pracownik  
ADD CONSTRAINT CK_Wiek CHECK (wiek >= 18);  
Nie pozwoli dodać pracownika młodszego niż 18 lat.
```

9. INDEX (przyspieszenie wyszukiwania)

```
ALTER TABLE Pracownik ADD INDEX idx_WIEK (wiek);
```

10. Zmiana nazwy kolumny

```
ALTER TABLE Pracownik  
CHANGE COLUMN imie imie_pracownika VARCHAR(50);
```

11. Zmiana nazwy tabeli

`RENAME TABLE` osoba **TO** osoby;

12. Zmiana kolejności kolumn (bardzo ważne)

Możesz ustawić, gdzie ma się pojawić kolumna w tabeli:

12.1 na początek tabeli

```
select * from pracownicy;  
ALTER TABLE Pracownicy MODIFY COLUMN wiek INT FIRST;  
select * from pracownicy;
```

12.2 po konkretnej kolumnie

```
select * from pracownicy;  
ALTER TABLE Pracownicy  
MODIFY COLUMN wiek INT AFTER id;  
select * from pracownicy;
```

13. Zmiana nazwy kolumny + typu

`ALTER TABLE` Pracownicy **CHANGE COLUMN** wiek wiek_pracownika **INT**;
używane gdy zmieniasz „model danych”

14. Dodanie AUTO_INCREMENT

Bardzo często w ID:

`ALTER TABLE` Pracownicy **MODIFY COLUMN** id **INT AUTO_INCREMENT**;

15. Usuwanie kluczy obcych (FOREIGN KEY)

```
ALTER TABLE Pracownicy  
DROP FOREIGN KEY FK_Pracownik_Dzial;
```

16. Usuwanie indeksów

```
ALTER TABLE Pracownicy  
DROP INDEX idx_email;
```

17. Dodanie wielu kolumn na raz

```
ALTER TABLE Pracownicy  
ADD COLUMN email VARCHAR(100),  
ADD COLUMN telefon VARCHAR(20);
```

18. Dodanie wielu constraintów

```
ALTER TABLE Pracownik  
ADD CONSTRAINT CK_Wiek CHECK (wiek >= 18)
```


